

MLOps Demo 01: Data & Pipeline Foundations

Instructor Demo Notes

Goal

Walk through a minimal, reproducible MLOps workflow:

1. Create a Python virtual environment.
2. Initialize a Git + DVC repository.
3. Build a simple data pipeline in Python.
4. Run basic data validation.
5. Version the processed dataset with DVC.
6. Simulate a data/schema change and show how validation and DVC help.

This is the exact sequence to use in class, live on your machine.

1 Environment & Repository Setup

1.1 Create project folder and virtual environment

1. Open a terminal and create the project directory:

```
mkdir mlops-intro-demo  
cd mlops-intro-demo
```

2. Create a virtual environment:

```
python -m venv .venv
```

‘.venv’ means by default hidden and telling everyone that its a internal project folder, not actual code etc. ‘ls -la’ lists all folders, visible or hidden in the folder.

3. Activate the virtual environment:

macOS / Linux:

```
source .venv/bin/activate
```

Windows (PowerShell):

```
.venv\Scripts\Activate.ps1
```

Always make sure to activate the virtual environment before installing packages.

1.2 Install packages and initialize Git + DVC

1. Install required Python packages:

```
pip install pandas scikit-learn dvc
```

2. Freeze them into requirements.txt:

```
pip freeze > requirements.txt
```

3. Initialize Git and DVC:

```
git init  
dvc init
```

4. Create the basic folder structure:

```
mkdir -p data/raw data/processed src
```

raw to hold the raw data, processed to hold the cleaned, validated and processed data and src to hold the source codes for the pipeline

2 Prepare a Small Raw Dataset

1. Create a CSV file at data/raw/customers.csv with content similar to:

```
customer_id,age,income,churned  
1,25,42000,0  
2,40,78000,1  
3,19,25000,0  
4,55,90000,0  
5,33,61000,1
```

2. Stage and commit the initial repo state:

```
git add .  
git commit -m "Initial repo with venv, raw data, and DVC"
```

3 Build the Data Pipeline Script

3.1 Create `src/pipeline.py`

1. Open `src/pipeline.py` and paste the following:

```
import pandas as pd
from pathlib import Path

RAW_PATH = Path("data/raw_data/customers.csv")
PROCESSED_PATH = Path("data/processed/customers_clean.csv")

def load_data(path: Path) -> pd.DataFrame:
    return pd.read_csv(path)

def clean_data(df: pd.DataFrame) -> pd.DataFrame:
    # simple cleaning: drop NA, filter invalid ages
    df = df.dropna(subset=["age"])
    df = df[df["age"].between(18, 90)]
    return df

def validate_data(df: pd.DataFrame) -> None:

    # make sure at least one income column is there
    has_income = "income" in df.columns
    assert has_income, "No income column found"

    has_age = "age" in df.columns
    assert has_age, "No age column found"

def save_data(df: pd.DataFrame, path: Path) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    df.to_csv(path, index=False)

def main():
    df = load_data(RAW_PATH)
    validate_data(df)
    clean_df = clean_data(df)
    save_data(clean_df, PROCESSED_PATH)
    print(f"Saved processed data to {PROCESSED_PATH}")

if __name__ == "__main__":
```

```
main()
```

2. Run the pipeline code, while in the `mlops-intro-demo` folder (ensure the virtual environment is still active):

```
python src/pipeline.py
```

3. Confirm that `data/processed/customers_clean.csv` was created.

4 First Data Version with DVC

4.1 Track processed dataset

1. Ask DVC to track the processed dataset:

```
dvc add data/processed/customers_clean.csv
```

2. Stage the generated files:

```
git add data/processed/customers_clean.csv.dvc data/processed/.gitignore
```

Telling git to store the checksum file to the dataset, not the actual data, because that would be too large. DVC tracks the actual data file. Also get the gitignore file.

3. Commit:

```
git commit -m "Add first processed dataset version tracked by DVC"
```

4. Check current DVC status:

```
dvc status
```

It should report that everything is up to date.

4.2 Explain DVC concept (for class)

Key talking points you can use while showing the files:

- Git is excellent for code and small text files, but not for large or frequently changing datasets.
- DVC stores the *data* separately and keeps a small pointer file (the `.dvc` file) under Git.
- By committing the `.dvc` file, we can always recover exactly which dataset version was used.

Useful commands:

```
# See what has changed since last commit (data-wise)
dvc status
```

```
# See Git history of the dataset pointer
git log --oneline data/processed/customers_clean.csv.dvc
```

5 Simulate a Data/Schema Change and Failure

5.1 Break the schema in the raw data

1. Open `data/raw/customers.csv` and modify it to break the schema. For example, rename `income` to `annual_income`:

```
customer_id,age,annual_income,churned
1,25,42000,0
2,40,78000,1
3,19,25000,0
4,55,90000,0
5,33,61000,1
```

2. Run the pipeline again:

```
python src/pipeline.py
```

3. You should see an assertion error similar to:

```
AssertionError: No income column found
```

This demonstrates how validation catches schema changes.

5.2 Observe DVC status after change

1. Run:

```
dvc status
```

The underlying raw data file has changed, but we have not yet recorded a new processed dataset version with DVC because the pipeline has failed. So DVC does not yet know anything!

6 Update Validation & Create a New Data Version

6.1 Relax / update validation logic

1. Update the `validate_data` function to allow the new schema:

```
def validate_data(df: pd.DataFrame) -> None:
    # allow either `income` or `annual_income`, but require one of them
    has_income = "income" in df.columns or "annual_income" in df.columns
    assert has_income, "No income column found"

    assert "age" in df.columns, "Missing age column"
```

2. Also update the cleaning function if needed (for example, unify the income column name internally). For the demo you can keep it simple.

6.2 Run pipeline and re-version the dataset

1. Run the pipeline again:

```
python src/pipeline.py
```

2. Check dvc status:

```
dvc status
```

Now you will see dvc has detected a changed from its older version to this new version.

3. Tell DVC to update tracking of the processed dataset:

```
dvc add data/processed/customers_clean.csv
```

4. Stage the data file:

```
git add data/processed/customers_clean.csv.dvc
```

5. Commit the changes:

```
git commit -am "Update validation for new schema and new processed dataset"
```

6.3 Inspect history

1. Show Git history for the dataset pointer:

```
git log --oneline data/processed/customers_clean.csv.dvc
```

2. (Optional) Show a diff between the last two commits:

```
dvc diff commitid1 commitid2
```

This reinforces that both code and data have versions and history. Get the commitids from the git log command

7 Quick Command Summary

- Virtual env:

```
python -m venv .venv  
source .venv/bin/activate    # or .venv\Scripts\Activate.ps1 on Windows
```

- Install + freeze:

```
pip install pandas scikit-learn dvc  
pip freeze > requirements.txt
```

- Git + DVC init:

```
git init  
dvc init
```

- Run pipeline:

```
python src/pipeline.py
```

- Track dataset with DVC:

```
dvc add data/processed/customers_clean.csv  
git add data/processed/customers_clean.csv.dvc data/.gitignore  
git commit -m "Message..."
```

- Check status and history:

```
dvc status  
git log --oneline data/processed/customers_clean.csv.dvc
```

Finally: Git and DVC work together to version code and data in a clean, scalable way. Git is used to track code and small metadata files, while DVC is used to track large datasets that should not be stored directly in Git. When a dataset changes, DVC stores the new version in its cache or remote storage and updates a .dvc file with a checksum that uniquely identifies that version. The .dvc file is then committed to Git, allowing Git history to act as an index of data versions. Each Git commit corresponds to exactly one version of the dataset through the .dvc file it contains. To restore an older version of the data, you check out the Git commit associated with that version. This restores the correct .dvc file and checksum. Running `dvc checkout` then retrieves the exact dataset version from DVC storage using that checksum.